Plan de implementación de las issues de GitHub

Fecha: 2026-09-08. Repositorio: `luistorres-idt/puk`.

> **Estado del documento (2026-10-02):** la tabla final registra verificación local histórica de las issues #4–#10; no refleja su estado actual en GitHub ni un despliegue vigente. Las funcionalidades enumeradas están implementadas en la rama de trabajo actual según contratos/código. Staging AWS se eliminó después de aquellas pruebas; no hay endpoint de staging activo. Los status remotos no se cambiaron ni se volvieron a consultar al actualizar esta documentación.

Orden de entrega

1. #7: contrato del libro y decisiones funcionales.
2. #6: directorio técnico sincronizado.
3. #4: técnico obligatorio al crear órdenes.
4. #8: servicios nuevos y política de altas posteriores.
5. #5: snapshot de precios.
6. #9: publicación del cobro.
7. #10: validación integral del épico.

Las validaciones de técnico activo de #4 y el enum, las etiquetas y las restricciones de #8 pueden adelantarse. La ejecución específica de los servicios nuevos y los cobros requieren las decisiones pendientes de #7. No se considera terminada una issue por haber implementado solamente esa parte independiente.

#7 - Contrato del libro

Fuente: https://github.com/luistorres-idt/puk/issues/7.

- Mantener la especificación versionada en `docs/technician-workbook-contract.md`, distinguiendo requisitos de las issues de datos verificados contra el libro.
- Obtener el libro de desarrollo/staging, encabezados y clave externa estable; verificar encabezados antes de importar o escribir. No utilizar números de fila como identidad ni inferir identidades por nombre, correo o teléfono.
- Acordar moneda, separadores, fecha, semana, zona horaria, unidad de cobro, congelación de tarifas, viáticos, citas fallidas y correcciones.
- Confirmar los flujos de Sensor fuel/Cambio de equipo; `nombre` y `rfc` quedan explícitamente fuera de la escritura de la aplicación y bajo captura manual.
- Preparar fixtures ficticios con vacíos, cero, duplicados, filas movidas y errores.

- Documentar configuración por ambiente. Validar permisos en staging sin escribir datos de prueba en producción.

Aceptación: contrato revisado con el negocio y prueba de acceso al libro de staging. Pendiente externo: libro, encabezados reales y reglas aún no confirmadas.

#6 - Directorio técnico

Fuente: https://github.com/luistorres-idt/puk/issues/6.

- Añadir un parser de directorio detrás del puerto de Sheets y el circuit breaker existentes. Separar lectura remota, validación y persistencia.
- Agregar clave externa única nullable para técnicos históricos, datos de contacto, facturación, importes `Decimal`, validez y metadatos del origen.
- Sincronizar transaccionalmente por clave externa. Rechazar duplicados antes de mutar; preservar UUIDs, enlaces a usuarios e historial. Desactivar retirados.
- Definir el enlace inicial de registros históricos con claves aprobadas, evitando asociaciones automáticas ambiguas por nombre.
- Exponer un endpoint específico de selección con proyección explícita de UUID, nombre, taller y ciudad. No devolver datos fiscales ni tarifas.
- Incorporar caché con caducidad y recarga explícita. Una caída de Sheets debe mostrarse y no interpretarse como catálogo vacío ni desactivar todos los técnicos.

Verificación: parser, decimales, duplicados, importación idempotente, conservación de UUID/enlaces, retirados, errores, expiración y recarga. Depende de #7.

#4 - Técnico obligatorio

Fuente: https://github.com/luistorres-idt/puk/issues/4.

- Exigir UUID en `CreateOrderSchema` y verificar existencia/actividad en el caso de uso. Impedir también quitar el técnico mediante una actualización del borrador.
- Conectar la validación de selección con el directorio sincronizado de #6 cuando esté disponible, para rechazar técnicos retirados entre selección y envío.
- Mantener nullable la relación en PostgreSQL para preservar lectura histórica. Auditar órdenes sin técnico, regularizar individualmente con asignaciones verificadas y registrar auditoría. No asignar técnicos por defecto masivamente.
- Mostrar carga, error, catálogo vacío y reintento en el wizard. Exigir selección vigente antes del resumen y del envío, y mostrar el nombre seleccionado.
- Actualizar OpenAPI, fixtures, pruebas del servicio, API y recorrido del wizard.

Aceptación: la API rechaza ausente/null/UUID inválido/inexistente/inactivo; una creación válida conserva técnico desde `DRAFT`. La exigencia de origen Sheets depende de #6.

#8 - Servicios y altas posteriores

Fuente: https://github.com/luistorres-idt/puk/issues/8.

- Agregar `FUEL_SENSOR` y `EQUIPMENT_REPLACEMENT` mediante migración aditiva.
- Centralizar etiquetas y opciones tipadas de los seis servicios para frontend, filtros, PDF y contratos. Mantener exhaustividad comprobada por TypeScript.
- Compartir la política de altas posteriores: cualquier servicio configurado mientras la orden permanezca en `DRAFT`. Aplicarla a los botones y endpoints de alta.
- Conservar lectura, edición y eliminación existentes. La carga inicial del wizard sigue entrando por `POST /orders` con unidades/equipos anidados.
- Implementar los flujos aprobados: sensor fuel usa equipo de stock y datos manuales de unidad; cambio de equipo vincula un equipo instalado y su reemplazo de stock.

Verificación: matriz por servicio/estado, endpoints de alta, creación inicial de revisión/desinstalación, visibilidad de componentes, etiquetas, OpenAPI y migración en una base aislada. El flujo nuevo depende de #7.

#5 - Snapshot de tarifa

Fuente: https://github.com/luistorres-idt/puk/issues/5.

- Crear un resolvedor puro y exhaustivo de servicio a columna de tarifa.
- Usar aritmética decimal y persistencia `Decimal`, moneda y redondeo documentados.
- Persistir identidad/nombre del técnico, servicio, costo base, salida del taller, viáticos aplicados, total, fecha de captura y versión del origen.
- Calcular/recalcular dentro de la transacción del cierre permitido y auditar el cambio. Congelar al pasar a `COMPLETED`; una corrección genera una versión nueva.
- Distinguir tarifa ausente de formato inválido y cero explícito. Emitir errores accionables y métricas sin datos personales ni payloads fiscales.
- Tratar correcciones mediante un flujo explícito con versión y auditoría; jamás sobrescribir históricos por una actualización del directorio.

Verificación: los seis servicios, taller/domicilio, redondeo, nulos, cero, inválidos, reintentos concurrentes y cambios posteriores en Sheets. Depende de #7, #6 y #8.

#9 - Reporte técnico de cobro

Fuente: https://github.com/luistorres-idt/puk/issues/9.

- Ampliar el outbox existente con una unión discriminada por `operationType` y payloads validados. Reutilizar worker, circuit breaker, reintentos y métricas.
- Encolar en la misma transacción del cierre cobrable, con clave única por orden y versión. La publicación remota nunca revierte el cierre confirmado.
- Serializar escrituras al reporte por libro/folio, incluyendo exclusión entre procesos. Buscar por folio, rechazar duplicados y volver a verificar identidad antes de escribir. No asumir que

un número de fila sigue identificando la orden.

- Escribir solo las columnas administradas por la aplicación, preservando `D` (`nombre`), `E` (`rfc`) y `O` (`envio dhl`) en inserciones, actualizaciones y reintentos. Validar encabezados y datos antes de la primera escritura.
- Generar celdas multilínea en orden estable y alinear económicos/series; agrupar equipos por unidad con secuencia documentada.
- Mostrar sincronización de cobro separada de equipos y permitir reintentos observables con errores sanitizados.

Verificación: inserción/actualización/reintento, O intacta, filas movidas, duplicados, encabezados inválidos, concurrencia entre workers, timeout después de escritura, caída de Sheets y correcciones. Depende de #7 y #5.

#10 - Épico

Fuente: https://github.com/luistorres-idt/puk/issues/10.

- Integrar las seis entregas sin duplicar lógica de dominio ni crear workers paralelos para el mismo mecanismo de outbox.
- Ejecutar lint, typecheck, pruebas, validación OpenAPI, build y E2E representativos.
- Probar migraciones sobre datos históricos en una base aislada.
- Validar el recorrido completo con el libro de staging y registrar evidencia de un solo cobro por folio y conservación de DHL.
- Marcar cada criterio por evidencia real; mantener pendientes los que requieran confirmación del negocio o validación externa.

Límites de implementación

Las reglas contables y los mapas de las issues #7 y #8 ya fueron confirmados e implementados. La validación automatizada usa datos ficticios; cualquier prueba manual adicional debe realizarse en el libro de staging, nunca en producción. Permanecen fuera de alcance los pagos a técnicos y la escritura de las columnas manuales `D`, `E` y `O` del reporte de cobro.

Estado de esta entrega

Actualización del 2026-09-10: se implementaron y verificaron las issues #4, #5, #6, #7, #8, #9 y #10 conforme a los contratos y decisiones acordadas.

Issue	Implementado y verificado localmente	Estado en GitHub
#7	Contrato del libro v1.0 en `technician-workbook-contract.md`, 88 identidades `TEC-0001` a `TEC-0088` verificadas en staging, reglas de tarifas, cálculo y exclusión de columnas D, E y O	Sin modificar status en GitHub (pendiente de autorización explícita)

Issue	Implementado y verificado localmente	Estado en GitHub
#6	Directorio técnico: parser A1:W1, IDs `TEC-XXXX`, sincronización transaccional y automática con TTL/fallback, tarifas, desactivación de retirados y selector con filtros de ciudad/taller	Sin modificar status en GitHub (pendiente de autorización explícita)
#4	Técnico obligatorio: validado en creación (`POST /orders`) y actualización (`PATCH /orders/:id`), wizard con loading/error/reintento, columna en Postgres nullable para compatibilidad histórica, snapshots de técnico persistidos	Sin modificar status en GitHub (pendiente de autorización explícita)
#8	Seis servicios habilitados, varios servicios por orden mediante `OrderUnit.serviceType`, altas en cualquier borrador y flujos finales de `SALIDAS` para instalación, revisión, sensor fuel, desinstalación y cambio de equipo	Sin modificar status en GitHub (pendiente de autorización explícita)
#5	Snapshot de tarifas: módulo puro `pricing.ts`, aritmética `Prisma.Decimal`, redondeo half-up a 2 decimales en MXN, base por equipo (1 para reemplazo), viáticos únicos en servicio a domicilio, fallback a 0.00 en cita fallida sin tarifa y congelación al completar	Sin modificar status en GitHub (pendiente de autorización explícita)
#9	Reporte técnico: formateador `technical-report-formatter.ts` con fecha/semana en `America/Hermosillo` y multilíneas en F, G, H; outbox discriminado `SHEET_TECHNICAL_REPORT`, reintentos, cliente Google Sheets escribiendo estrictamente A:C, F:N y P (preservando D, E y O intactas), reabierto y recálculo en corrección por ADMIN	Sin modificar status en GitHub (pendiente de autorización explícita)
#10	Épico integrado: outbox unificado con métricas por `operationType`, circuit breaker decorando nuevos métodos, auditoría completa de transacciones, validación OpenAPI sin errores y suite de pruebas unitarias/integración	Sin modificar status en GitHub (pendiente de autorización explícita)

No se modificó el estado de las issues en GitHub, conforme a las instrucciones del usuario.

Evidencia de validación

- `pnpm typecheck`: 6 proyectos de workspace verificados exitosamente (`@puk/contracts`, `@puk/api`, `@puk/web`, `infra/cdk`, etc.) con 0 errores.
- `pnpm test`: 202 pruebas pasando en `@puk/api` (28 archivos, más 2 archivos de integración omitidos), 22 en `@puk/contracts` y 40 en `@puk/web`.
- `pnpm docs:validate`: Documento OpenAPI generado y validado contra especificaciones sin discrepancias.
- Migraciones Prisma: además del directorio y snapshots, `20260909174050_multi_service_orders_and_unit_service_type` agrega el desglose de precios y el tipo por unidad con backfill desde la orden.

- Reglas de secreto y seguridad: Redacción activa en métricas y logs para datos sensibles, fiscales y bancarios; nunca se toca producción ni se exponen credenciales.